



HTML FORM & PHP KONEKSI



SISTEM INFORMASI
UNIVERSITAS TRUNOJOYO MADURA

MODUL PEMROGRAMAN BERBASIS WEB DASAR

Ir. Hanifudin Sukri, S.Kom., M.Kom.

Dr. Imamah, S.Kom., M.Kom.

MODUL 6**HTML FORM & PHP KONEKSI****Tujuan :**

- Mahasiswa mampu memahami HTML form dan koneksi.
- Mahasiswa mampu memahami koneksi PHP dengan database.
- Mahasiswa mampu memahami Create, Read, Update, dan Delete

Tugas Pendahuluan:

1. Jelaskan perbedaan antara method **GET** dan **POST** pada form HTML. Berikan satu contoh kasus penggunaan yang lebih cocok untuk masing-masing method tersebut.
2. Apa yang dimaksud dengan serangan **XSS (Cross-Site Scripting)**, dan bagaimana fungsi `htmlspecialchars()` dalam PHP membantu mencegahnya? Jelaskan pula mengapa fungsi ini **tidak dapat** digunakan untuk mencegah serangan SQL Injection.
3. Saat sebuah form HTML di-submit ke file PHP untuk disimpan ke database, jelaskan secara berurutan langkah-langkah yang dilakukan PHP, mulai dari menerima data form hingga data tersimpan ke database. Sebutkan minimal lima langkah utama.
4. Pada operasi Delete dalam aplikasi web, ID data yang ingin dihapus biasanya dikirimkan melalui URL (contoh: `hapus.php?id=5`). Jelaskan alasan umum mengapa pengiriman ID untuk operasi Delete menggunakan method **GET**. Sebutkan juga minimal **dua hal** yang perlu dilakukan di sisi PHP saat menerima parameter id dari URL agar tidak terjadi error atau masalah saat data diproses.
5. Jelaskan perbedaan antara fungsi `include`, `require`, `include_once`, dan `require_once` di PHP. Mengapa untuk file koneksi database lebih disarankan menggunakan `require_once` dibandingkan `include`?

A. HTML FORM DALAM PHP

1. Apa Itu Html Form

HTML form adalah bagian dari halaman web yang memungkinkan pengguna mengirimkan data ke server. Bayangkan form seperti kotak isian di halaman web. pengguna mengetikkan informasi seperti nama, alamat email, atau pesan, lalu menekan tombol untuk mengirimkannya.

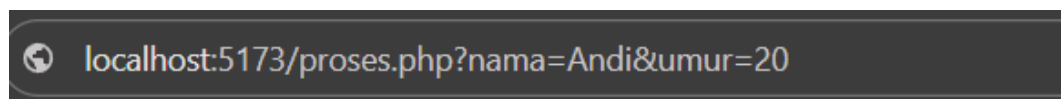
Sebelumnya kalian sudah belajar membuat tag `<form>` beserta input-input dasarnya seperti `text`, `radio`, `checkbox`, dan `submit`. Nah, sekarang kita fokus ke langkah berikutnya yaitu **bagaimana data form itu diproses oleh PHP**, lalu ditampilkan ke pengguna atau disimpan ke database.

2. Method GET dan POST

Setiap form HTML punya atribut `method` yang menentukan **cara data dikirim ke server**. Ada dua method utama: GET dan POST. Memahami perbedaannya penting karena salah pilih bisa membuat aplikasi tidak aman atau tidak berfungsi sebagaimana mestinya.

2.1 Method GET

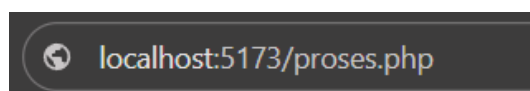
data dikirim lewat URL, terlihat oleh siapa saja yang melihat alamat browser. Misalnya kalau form pakai `method="get"`, setelah submit URL-nya menjadi:



Data `nama=Andi` dan `umur=20` "menempel" di URL. Konsekuensinya: data bisa di-bookmark, panjangnya terbatas (sekitar 2000 karakter), dan tidak cocok untuk informasi sensitif karena terekam di history browser dan log server.

2.2 Method POST

data dikirim "tersembunyi" di body HTTP request, tidak terlihat di URL. Setelah submit, URL tetap bersih:



POST cocok untuk data dalam jumlah besar, data sensitif, dan operasi yang mengubah keadaan database (insert, update, delete).

Kapan pakai yang mana?

Pakai GET kalau....

Pakai POST kalau....

Data tidak sensitif	Data sensitif (password, info pribadi)
User mungkin ingin bookmark hasilnya	Form mengubah data di database
Fitur pencarian (?cari=sepatu)	Upload file
Filter halaman (?kategori=elektronik)	Login dan registrasi

Cara mengakses datanya di PHP juga berbeda. Kalau form pakai GET, di PHP kita ambil pakai `$_GET['nama_input']`. Kalau POST, pakai `$_POST['nama_input']`.

3. Cara Membuat Html Form

Mari kita buat form sederhana untuk mengumpulkan nama, umur, dan kota pengguna:

Index.php

```
<!DOCTYPE html>
<html lang="id">
<head>
  <title>Form Data Pengguna</title>
</head>
<body>
  <h2>Formulir Data Pengguna</h2>
  <form action="proses.php" method="POST">
    <div>
      <label>Nama:</label><br>
      <input type="text" name="nama" required>
    </div>
    <br>
    <div>
      <label>Umur:</label><br>
      <input type="number" name="umur" required>
    </div>
    <br>
    <div>
      <label>Kota:</label><br>
      <select name="kota">
        <option value="Jakarta">Jakarta</option>
        <option value="Bandung">Bandung</option>
        <option value="Surabaya">Surabaya</option>
      </select>
    </div>
    <br>
  </form>
</body>
```

```
<button type="submit" name="submit">Kirim Data</button>
</form>
</body>
</html>
```

Beberapa hal yang perlu diperhatikan:

- **action="proses.php"** file tujuan yang akan menerima dan memproses data.
 - **method="POST"** cara pengiriman, sesuai diskusi sebelumnya.
 - Atribut **name** di setiap input adalah **kunci penting**. Nilai inilah yang nanti dipanggil di PHP via **\$_POST['nama']**. Tanpa atribut **name**, data input tidak akan terkirim ke server.
 - **required** atribut HTML5 yang mencegah form di-submit kalau field kosong.
- Ini validasi sederhana di sisi browser.

4. PHP Mengolah Data Form

Setelah pengguna menekan tombol "Kirim Data", data form dikirim ke **proses.php**. Sekarang giliran PHP yang menanganinya. Berikut isi file **proses.php**:

```
<?php
// Cek apakah halaman ini diakses lewat submit form (POST)
if ($_SERVER["REQUEST_METHOD"] == "POST") {

    // Ambil data dari form, bersihkan untuk tampilan (anti-XSS)
    $nama = htmlspecialchars($_POST['nama']);
    $umur = htmlspecialchars($_POST['umur']);
    $kota = htmlspecialchars($_POST['kota']);

    // Tampilkan hasil ke pengguna
    echo "<h2>Konfirmasi Data</h2>";
    echo "Terima kasih, <b>$nama</b>.<br>";
    echo "Kamu yang berumur $umur tahun tinggal di kota $kota.";

    echo "<br><br><a href='index.php'>Kembali ke Form</a>";
} else {
    // Kalau seseorang buka proses.php langsung tanpa lewat form
    echo "Silakan isi form terlebih dahulu!";
}
?>
```

Penjelasan baris demi baris:

1. `$_SERVER["REQUEST_METHOD"] == "POST"`: kita cek dulu apakah file ini diakses karena ada form yang di-submit, atau seseorang mengetik URL `proses.php` langsung di browser. Kalau diakses langsung, tampilkan pesan agar mereka isi form dulu. Ini praktik kecil tapi penting untuk mencegah error.
2. `$_POST['nama']`: cara mengambil nilai dari input yang punya `name="nama"`. Pola ini berlaku untuk semua input dengan method POST. Untuk method GET, ganti `$_POST` jadi `$_GET`.
3. `htmlspecialchars(...)`: fungsi keamanan yang mengubah karakter HTML berbahaya seperti `<`, `>`, `"`, dan `&` menjadi versi "aman" (`<`, `>`, `"`, `&`). Ini melindungi halaman dari serangan XSS (Cross-Site Scripting), kalau ada pengguna nakal yang mencoba memasukkan `<script>alert('hacked')</script>` sebagai nama, fungsi ini akan menampilkannya sebagai teks biasa, bukan menjalankannya sebagai script.

Apa yang terjadi ketika form di-submit? Saat pengguna mengklik tombol "Kirim Data":

1. Browser mengumpulkan semua nilai input yang punya atribut `name`.
2. Browser mengirim data tersebut ke URL yang ditentukan di `action` (yaitu `proses.php`), dengan method yang ditentukan di `method` (POST).
3. Server menjalankan `proses.php`. Di file ini, PHP punya akses ke data tadi lewat array global `$_POST`.
4. PHP mengolah data dan mengirim balik HTML ke browser untuk ditampilkan.

B. PHP KONEKSI DATABASE

1. Konsep Koneksi Database

Koneksi database adalah cara bagi program PHP untuk "berbicara" dengan database. Bayangkan database seperti rak buku besar yang berisi banyak informasi tersusun rapi. Kita tidak bisa langsung mengambil buku dari rak, kita butuh seorang petugas perpustakaan sebagai perantara. Dalam analogi ini, **PHP adalah petugas perpustakaan** yang menerima permintaan dari halaman web, mengambil atau menyimpan data ke database, lalu mengembalikan hasilnya ke pengguna.

Alur kerjanya sederhana:

1. Pengguna membuka halaman web atau mengisi form.
2. PHP menerima permintaan dan **membuka koneksi** ke database.
3. PHP mengirim "pertanyaan" (query) ke database misalnya "tampilkan semua data guru".
4. Database menjalankan query dan mengembalikan hasilnya ke PHP.
5. PHP menyajikan hasil tersebut sebagai HTML ke browser pengguna.

Tanpa koneksi, PHP dan database hanyalah dua sistem terpisah yang tidak saling kenal.

2. Mengapa Kita Butuh Database?

Sebelumnya kalian sudah belajar menyimpan data di **variabel** dan **array** PHP. Tapi data di sana punya kelemahan besar: **data hilang saat halaman selesai dimuat atau server di-restart**. Coba bayangkan kalau aplikasi e-commerce menyimpan daftar produk di array PHP saja, setiap kali server restart, semua produk hilang. Setiap kali pengguna refresh halaman, data form yang baru diisi lenyap.

Database memecahkan masalah ini, ia menyimpan data secara **persisten** (tidak hilang sampai sengaja dihapus), bisa diakses oleh banyak pengguna sekaligus, dan punya mekanisme untuk mencari, memfilter, serta mengelola data dalam jumlah besar dengan efisien. Kita akan menggunakan **MySQL** sebagai database, dan **mysqli** sebagai library PHP yang menghubungkan keduanya.

3. Membuat Koneksi Database di PHP

Pertama-tama, pastikan **XAMPP** sudah berjalan Apache dan MySQL aktif di Control Panel. Kalau kalian pakai **Laragon** sebagai alternatif, prinsipnya sama: aktifkan Apache dan MySQL dari panel Laragon. Buka phpMyAdmin di browser (<http://localhost/phpmyadmin>) untuk memastikan database server bisa diakses.

Berikut kode untuk membuat koneksi PHP ke MySQL. Simpan sebagai file bernama **koneksi.php**:

koneksi.php

```
<?php
// Informasi server database
$servername = "localhost"; // alamat server (default)
$username   = "root";      // username default
$password   = "";          // password kosong (default)
```

```
$database = "db_modul6"; // nama database yang akan kita
pakai

// Membuat koneksi
$conn = new mysqli($servername, $username, $password,
$database);

// Mengecek koneksi
if ($conn->connect_error) {
    die("Koneksi ke database gagal: " . $conn->connect_error);
}

// Kalau sampai ke baris ini, koneksi berhasil
// echo "Koneksi berhasil!"; // hapus tanda komentar untuk
testing
?>
```

Penjelasan:

1. **Empat variabel di atas** adalah informasi yang dibutuhkan untuk login ke database server: server-nya di mana, login pakai username apa, password-nya apa, dan database mana yang ingin diakses. Untuk XAMPP yang baru di-install, default-nya selalu username **root** dengan password kosong ("").
2. **new mysqli(...)** membuat objek koneksi baru. Kalau berhasil, objek **\$conn** inilah yang nanti dipakai untuk semua operasi database (insert, select, update, delete).
3. **\$conn->connect_error** mengecek apakah ada error saat koneksi. Kalau ada (misalnya MySQL belum dijalankan, atau nama database salah), fungsi **die()** akan menghentikan eksekusi dan menampilkan pesan error. Tanpa pengecekan ini, error koneksi menyebabkan halaman blank tanpa informasi apa-apa sangat menyulitkan saat debugging.
4. Baris **echo "Koneksi berhasil!"** sengaja dikomentari. Aktifkan sementara saat testing pertama kali untuk memastikan koneksi jalan, lalu komentari lagi. Kalau dibiarkan aktif, pesan ini akan muncul di setiap halaman yang memakai koneksi dan mengganggu tampilan.

 **TIPS:** Sebelum menjalankan kode ini, pastikan database db_modul6 sudah dibuat di phpMyAdmin. Pembuatan database dan tabelnya akan dibahas di

bawah.

4. Memisahkan Koneksi ke File Tersendiri

Kalau setiap kali butuh koneksi database harus menulis ulang semua kode di atas, akan muncul tiga masalah:

1. **Kode jadi panjang dan berulang.** Bayangkan kalau aplikasi/sistem punya 20 file harus copy-paste 20 kali.
2. **Kalau ada perubahan, repot.** Misalnya password database diubah, kita harus mencari dan mengubah di 20 file. Lupa satu error di mana-mana.
3. **Rentan typo.** Setiap kali ditulis ulang, peluang typo bertambah.

PHP punya solusi: **include**. Fungsi ini menyisipkan isi file lain ke posisi tempat kita memanggilnya seolah-olah kode dari file itu ditulis langsung di sana.

4.1 include vs require vs require_once

PHP sebenarnya punya beberapa varian fungsi untuk menyertakan file:

Fungsi	Perilaku saat file tidak ditemukan
<code>include 'file.php'</code>	Menampilkan warning , kode di bawahnya tetap dijalankan
<code>require 'file.php'</code>	Menampilkan fatal error , eksekusi dihentikan total
<code>include_once 'file.php'</code>	Sama seperti <i>include</i> , tapi mencegah file disertakan dua kali
<code>require_once 'file.php'</code>	Sama seperti <i>require</i> , tapi mencegah file disertakan dua kali

Untuk file koneksi database, sebenarnya **require_once** lebih aman. Alasannya:

- a) kalau file koneksi hilang, lebih baik aplikasi langsung berhenti daripada lanjut tanpa database (yang akan menyebabkan error berantai)
- b) **_once** mencegah error kalau tidak sengaja file disertakan berkali-kali.

Kali ini kita tetap pakai **include** agar konsisten dan lebih sederhana untuk pemula. Saat kalian membangun aplikasi/sistem nyata nanti, silakan langsung pakai **require_once**.

4.2 Struktur Folder yang Akan Kita Pakai

```
htdocs/
├── modul6/
│   ├── koneksi.php      ← file koneksi (dibuat sekali, dipakai semua)
│   ├── index.php        ← form input data
│   ├── proses.php       ← memproses input dari form
│   ├── tampil_data.php  ← menampilkan data dari database
│   ├── update.php       ← memperbarui data
│   └── hapus.php        ← menghapus data
```

Folder **htdocs** adalah lokasi default XAMPP (biasanya **C:\xampp\htdocs**). Kalau pakai Laragon, lokasinya di **C:\laragon\www**. Sesuaikan saja.

Setiap file PHP yang butuh akses database cukup dimulai dengan satu baris:

```
<?php
include 'koneksi.php';
// ... kode operasi database di sini
?>
```

C. OPERASI CRUD DENGAN DATABASE

Sebelum mulai, kita perlu menyiapkan database dan tabel di MySQL. Buka **phpMyAdmin** (<http://localhost/phpmyadmin>), klik tab **SQL**, lalu jalankan perintah berikut:

```
-- Membuat database baru
CREATE DATABASE IF NOT EXISTS db_modul6;
USE db_modul6;

-- Membuat tabel guru
CREATE TABLE guru (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nama VARCHAR(100) NOT NULL,
    umur INT NOT NULL
);

-- Tambahkan beberapa data awal untuk testing
INSERT INTO guru (nama, umur) VALUES
    ('Andi', 35),
    ('Budi', 42),
    ('Citra', 28);
```

Penjelasan:

- **CREATE DATABASE IF NOT EXISTS** membuat database hanya kalau

belum ada aman dijalankan berulang.

- **USE db_modul6** memilih database yang akan dipakai.
- Kolom **id** pakai **AUTO_INCREMENT PRIMARY KEY** artinya MySQL otomatis memberikan nomor unik untuk tiap baris baru. Kita tidak perlu mengisi **id** saat insert.
- Tiga data awal berfungsi supaya operasi Read langsung menampilkan sesuatu, tidak kosong.

1. Menyimpan Data Baru ke Database (CREATE)

Operasi Create adalah cara menambahkan data baru ke dalam database.

Misalnya: pengguna mengisi form, lalu kita simpan ke tabel.

Kita sudah membuat **index.php** (form) dan **proses.php** (yang hanya menampilkan kembali data). Sekarang kita **upgrade proses.php** supaya juga menyimpan data ke database.

Index.php

```
<!DOCTYPE html>
<html lang="id">
<head>
  <title>Tambah Data Guru</title>
</head>
<body>
  <h2>Form Tambah Data Guru</h2>
  <form action="proses.php" method="POST">
    <label>Nama:</label><br>
    <input type="text" name="nama" required><br><br>

    <label>Umur:</label><br>
    <input type="number" name="umur" required><br><br>

    <button type="submit" name="submit">Simpan</button>
  </form>

  <br>
  <a href="tampil_data.php">Lihat Daftar Guru</a>
</body>
</html>
```

proses.php

```
<?php
// 1. Panggil koneksi database
include 'koneksi.php';

// 2. Cek apakah form sudah di-submit
if (isset($_POST['submit'])) {

    // 3. Ambil data dari form
    $nama = $_POST['nama'];
    $umur = (int) $_POST['umur']; // dipaksa jadi integer

    // 4. Buat query INSERT
    $sql = "INSERT INTO guru (nama, umur) VALUES ('$nama',
'$umur')";

    // 5. Jalankan query dan cek hasilnya
    if ($conn->query($sql) === TRUE) {
        echo "<h3>Berhasil!</h3>";
        echo "Data guru bernama <b>$nama</b> sudah masuk ke
database.";
        echo "<br><br><a href='tampil_data.php'>Lihat Daftar
Guru</a>";
        echo " | <a href='index.php'>Tambah Data Lagi</a>";
    } else {
        echo "Gagal menyimpan: " . $conn->error;
    }

    // 6. Tutup koneksi
    $conn->close();
}
?>
```

Penjelasan langkah demi langkah:

1. `include 'koneksi.php'` menyertakan file koneksi yang dibuat sebelumnya. Setelah baris ini, variabel `$conn` siap digunakan.
2. `isset($_POST['submit'])` cek apakah tombol submit form sudah ditekan. Mencegah eksekusi kalau file dibuka langsung tanpa form.
3. `(int) $_POST['umur']` paksa konversi ke integer. Praktik kecil tapi penting: kalau user iseng kirim teks di field umur (lewat tools developer), `(int)` mengubahnya jadi 0, mencegah error SQL.

4. **INSERT INTO ... VALUES (...)** perintah SQL untuk menambahkan baris baru. Variabel PHP disisipkan langsung ke string SQL.
5. **\$conn->query(\$sql)** eksekusi query. Mengembalikan TRUE kalau berhasil, FALSE kalau gagal.
6. **\$conn->close()** menutup koneksi setelah selesai. Praktik baik untuk membebaskan resource server.

2. Menampilkan Data dari Database (READ)

Operasi Read adalah cara mengambil data dari database dan menampilkannya. Biasanya berupa daftar (list) yang ditampilkan dalam tabel HTML.

tampil_data_php

```
<?php
// 1. Panggil koneksi
include 'koneksi.php';

// 2. Query untuk mengambil semua data dari tabel guru
$sql = "SELECT * FROM guru";
$result = $conn->query($sql);
?>

<!DOCTYPE html>
<html>
<head>
    <title>Daftar Guru</title>
    <style>
        table { border-collapse: collapse; width: 60%; }
        th, td { border: 1px solid black; padding: 8px; text-align: left; }
        th { background-color: #f2f2f2; }
        .aksi a { margin-right: 8px; }
    </style>
</head>
<body>
    <h2>Daftar Guru</h2>
    <a href="index.php">+ Tambah Guru Baru</a><br><br>

    <?php if ($result->num_rows > 0): ?>
        <table>
```

```

        <tr>
            <th>ID</th>
            <th>Nama</th>
            <th>Umur</th>
            <th>Aksi</th>
        </tr>
    <?php
    // 3. Looping setiap baris data
    while ($row = $result->fetch_assoc()):
    ?>
        <tr>
            <td><?= $row['id']; ?></td>
            <td><?= $row['nama']; ?></td>
            <td><?= $row['umur']; ?></td>
            <td class="aksi">
                <a href="update.php?id=<?= $row['id'];
?>">Edit</a>
                <a href="hapus.php?id=<?= $row['id']; ?>"
                    onclick="return confirm('Yakin mau hapus
data ini?')">Hapus</a>
            </td>
        </tr>
    <?php endwhile; ?>
</table>
<?php else: ?>
    <p>Tidak ada data guru.</p>
<?php endif; ?>

<?php $conn->close(); ?>
</body>
</html>

```

Penjelasan poin penting:

1. **SELECT * FROM guru** perintah SQL mengambil semua kolom dan semua baris dari tabel guru.
2. **\$result->num_rows** properti yang berisi jumlah baris hasil query. Kita cek dulu sebelum mencoba menampilkan, agar kalau kosong tidak error.
3. **\$result->fetch_assoc()** mengambil satu baris dari hasil query sebagai array asosiatif. Dipakai dalam loop **while**, looping berhenti otomatis saat tidak ada baris lagi.

4. `<?= ... ?>` sintaks pendek untuk `<?php echo ... ?>`. Lebih ringkas saat dipakai di tengah HTML.
5. **Kolom Aksi** dengan tombol Edit dan Hapus, link membawa parameter **id** di URL (`update.php?id=3`). Inilah yang akan dipakai oleh `update.php` dan `hapus.php` untuk tahu data mana yang harus diolah.
6. `onclick="return confirm(...)"` JavaScript bawaan browser untuk dialog konfirmasi. Kalau user klik "Cancel", link tidak akan dijalankan.

3. Memperbarui Data yang Ada (UPDATE)

Operasi Update memperbarui data yang sudah ada di database. Polanya: user klik link "Edit" di daftar → muncul form yang sudah berisi data lama → user ubah → submit → data di database berubah.

Kita pakai **satu file** `update.php` yang menangani dua kasus:

- Saat dibuka via GET (`update.php?id=3`) → tampilkan form berisi data lama
- Saat di-submit via POST → simpan perubahan ke database

update.php

```
<?php
include 'koneksi.php';

// === BAGIAN 1: Saat form di-submit (POST) ===
if (isset($_POST['update'])) {
    $id    = (int) $_POST['id'];
    $nama  = $_POST['nama'];
    $umur  = (int) $_POST['umur'];

    $sql = "UPDATE guru SET nama='$nama', umur='$umur' WHERE
id='$id'";

    if ($conn->query($sql) === TRUE) {
        echo "<h3>Berhasil!</h3>";
        echo "Data guru dengan ID $id berhasil diperbarui.";
        echo "<br><a href='tampil_data.php'>Lihat Daftar</a>";
    } else {
        echo "Gagal memperbarui: " . $conn->error;
    }
}
$conn->close();
exit; // hentikan eksekusi, jangan lanjut ke form di bawah
```

```

}

// === BAGIAN 2: Saat dibuka via GET (tampilkan form pre-filled)
===
if (!isset($_GET['id'])) {
    die("ID tidak ditemukan!");
}

$id = (int) $_GET['id'];
$sql = "SELECT * FROM guru WHERE id='$id'";
$result = $conn->query($sql);

if ($result->num_rows == 0) {
    die("Data tidak ditemukan!");
}

$row = $result->fetch_assoc();
?>
<!DOCTYPE html>
<html>
<head><title>Edit Data Guru</title></head>
<body>
    <h2>Edit Data Guru</h2>
    <form action="update.php" method="POST">
        <input type="hidden" name="id" value="<?= $row['id'];
?>">

        <label>Nama:</label><br>
        <input type="text" name="nama" value="<?= $row['nama'];
?>" required><br><br>

        <label>Umur:</label><br>
        <input type="number" name="umur" value="<?=
$row['umur']; ?>" required><br><br>

        <button type="submit" name="update">Simpan
Perubahan</button>
        <a href="tampil_data.php">Batal</a>
    </form>
</body>

```



```
</html>
```

Penjelasan poin kunci:

1. Dua "bagian" dalam satu file, dibedakan oleh `isset($_POST['update'])` (cek apakah form di-submit). Ini pola umum di PHP yang menghemat file.
2. `exit` setelah bagian POST penting! Tanpa ini, setelah update sukses, PHP akan lanjut ke bagian GET dan menampilkan form lagi.
3. `<input type="hidden" name="id">` input tersembunyi yang membawa `id` ikut ter-submit, supaya bagian POST tahu data mana yang harus di-update.
4. `value="<?= $row['nama']; ?>"` mengisi otomatis form dengan data lama dari database. Inilah yang membuat user melihat data lama, bukan form kosong.
5. `WHERE id='$id'` di query UPDATE **WAJIB**. Tanpa `WHERE`, semua baris di tabel akan ter-update. Selalu pakai `WHERE` saat update data spesifik.

4. Menghapus Data (DELETE)

Operasi Delete menghapus baris dari database. Polanya: user klik link "Hapus" di daftar (yang membawa `id` di URL) → konfirmasi → data dihapus.

hapus.php

```
<?php
include 'koneksi.php';

// 1. Cek apakah parameter id ada di URL
if (!isset($_GET['id'])) {
    die("ID tidak ditemukan!");
}

// 2. Ambil id dari URL
$id = (int) $_GET['id'];

// 3. Jalankan query DELETE
$sql = "DELETE FROM guru WHERE id='$id'";

if ($conn->query($sql) === TRUE) {
    // 4. Setelah berhasil, langsung redirect kembali ke daftar
    header("Location: tampil_data.php");
    exit;
} else {
```

```

        echo "Gagal menghapus: " . $conn->error;
    }

    $conn->close();
?>

```

Penjelasan:

1. Cek `$_GET['id']` kalau seseorang membuka `hapus.php` langsung tanpa parameter, hentikan dengan pesan error.
2. `(int) $_GET['id']` paksa jadi integer. Ini juga bentuk pertahanan kecil terhadap input nakal di URL.
3. `DELETE FROM guru WHERE id='$id'` perintah SQL menghapus baris. **WHERE wajib** tanpa itu, seluruh tabel akan terhapus.
4. `header("Location: ...")` redirect browser ke halaman lain. Setelah delete sukses, langsung kembalikan user ke daftar agar mereka langsung melihat data yang hilang.
5. `exit` setelah `header()` best practice. Tanpa ini, kode di bawah masih dijalankan walaupun browser sudah dialihkan.

D. KEAMANAN & AUTENTIKASI

Sampai di sini, kalian sudah berhasil membuat aplikasi CRUD yang berfungsi. Tapi kode kalian sebenarnya masih punya dua kelemahan besar: **rentan terhadap serangan keamanan** dan **belum punya sistem login**. Bagian ini menyelesaikan keduanya.

1. SQL Injection dan Pencegahannya

SQL Injection adalah serangan di mana penyerang menyisipkan perintah SQL berbahaya melalui input pengguna. Penyebabnya: kode kita menyisipkan input langsung ke string SQL

```
$sql = "DELETE FROM guru WHERE id='$id'";
```

Selama `$id` berisi angka wajar, query baik-baik saja. Tapi penyerang bisa memanipulasi URL menjadi `hapus.php?id=' OR '1'=1`. Setelah PHP menggabungkan, query menjadi `DELETE FROM guru WHERE id=" OR '1'=1'`. Karena `'1'=1` selalu TRUE, seluruh data di tabel guru terhapus.

Solusinya: prepared statement. Cara ini memisahkan struktur query dari data input. Kita kirim "template" query dengan placeholder `?` ke MySQL, lalu mengisi

data ke placeholder secara terpisah. MySQL memperlakukan input sebagai data, bukan sintaks SQL.

Tiga langkah utama: **prepare** → **bind** → **execute**. Berikut versi aman dari **proses.php** (CREATE):

```
<?php
include 'koneksi.php';

if (isset($_POST['submit'])) {
    $nama = $_POST['nama'];
    $umur = (int) $_POST['umur'];

    // Versi prepared statement
    $stmt = $conn->prepare("INSERT INTO guru (nama, umur) VALUES
    (?, ?)");
    $stmt->bind_param("si", $nama, $umur);

    if ($stmt->execute()) {
        echo "Data berhasil disimpan!";
    } else {
        echo "Gagal: " . $stmt->error;
    }

    $stmt->close();
    $conn->close();
}
?>
```

Pada **bind_param("si", ...)**, karakter **s** artinya string dan **i** artinya integer. Pola yang sama berlaku untuk operasi lain UPDATE, DELETE, dan SELECT WHERE. Khusus untuk SELECT, gunakan **\$stmt->get_result()** setelah execute untuk mendapat objek hasil:

```
// SELECT WHERE
$stmt = $conn->prepare("SELECT * FROM guru WHERE id=?");
$stmt->bind_param("i", $id);
$stmt->execute();
$result = $stmt->get_result();
$row = $result->fetch_assoc();

// DELETE
```

```
$stmt = $conn->prepare("DELETE FROM guru WHERE id=?");
$stmt->bind_param("i", $id);
$stmt->execute();
```

2. Session dan Password Hashing

Session adalah cara PHP "mengingat" data antar request untuk pengguna yang sama. Saat user login, kita simpan informasinya di session, lalu setiap request berikutnya PHP mengenali user tanpa perlu login ulang. Empat perintah utama:

```
session_start(); // Mulai/lanjutkan session - WAJIB
di awal file
$_SESSION['user_id'] = 7; // Simpan data ke session
echo $_SESSION['user_id']; // Baca data dari session
session_destroy(); // Hancurkan session (logout)
```

Password Hashing. Password tidak boleh disimpan dalam bentuk asli di database. Kalau database bocor, semua password kebaca. Solusinya: hash password menjadi string acak yang tidak bisa dibalik.

Kolom database untuk menyimpan hash harus **VARCHAR(255)** karena hash bisa panjang dan formatnya bisa berubah di versi PHP berikutnya. PHP punya dua fungsi untuk ini:

```
// Saat REGISTER - simpan hash
$hash = password_hash("rahasia123", PASSWORD_DEFAULT);

// Saat LOGIN - verifikasi
if (password_verify($password_input, $hash_dari_database)) {
    // Password benar
}
```

3. Sistem Login Sederhana

Sekarang kita gabungkan semuanya. Mulai dari struktur tabel:

```
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    password VARCHAR(255) NOT NULL,
    role ENUM('admin', 'user') DEFAULT 'user'
);
```

File: register.php saat user mendaftar, password di-hash dulu sebelum disimpan. User baru otomatis dapat role user. Untuk membuat akun admin, ubah

role secara manual lewat phpMyAdmin:

```
<?php
include 'koneksi.php';

if (isset($_POST['daftar'])) {
    $username = $_POST['username'];
    $hash = password_hash($_POST['password'], PASSWORD_DEFAULT);

    $stmt = $conn->prepare("INSERT INTO users (username,
password) VALUES (?, ?)");
    $stmt->bind_param("ss", $username, $hash);
    $stmt->execute();

    echo "Registrasi berhasil! <a href='login.php'>Login</a>";
}
?>
<form method="POST">
    Username: <input type="text" name="username" required><br>
    Password: <input type="password" name="password"
required><br>
    <button type="submit" name="daftar">Daftar</button>
</form>
```

File: login.php verifikasi password lalu simpan info ke session. Pesan error sengaja generik ("Username atau password salah"), bukan spesifik seperti "username tidak ditemukan" agar penyerang tidak tahu username mana yang valid:

```
<?php
session_start();
include 'koneksi.php';

if (isset($_POST['login'])) {
    $stmt = $conn->prepare("SELECT * FROM users WHERE
username=?");
    $stmt->bind_param("s", $_POST['username']);
    $stmt->execute();
    $user = $stmt->get_result()->fetch_assoc();

    if ($user && password_verify($_POST['password'],
$user['password'])) {
        $_SESSION['user_id'] = $user['id'];
    }
}
```

```

        $_SESSION['username'] = $user['username'];
        $_SESSION['role']     = $user['role'];
        header("Location: tampil_data.php");
        exit;
    }
    echo "Username atau password salah.";
}
?>
<form method="POST">
    Username: <input type="text" name="username" required><br>
    Password: <input type="password" name="password"
required><br>
    <button type="submit" name="login">Login</button>
</form>

```

File: logout.php:

```

<?php
session_start();
session_destroy();
header("Location: login.php");
exit;
?>

```

4. Memproteksi Halaman dan Role-Based Access

Setelah punya sistem login, halaman seperti **tampil_data.php** perlu diproteksi agar hanya user yang sudah login yang bisa akses. Daripada copy-paste pengecekan di setiap file, buat **satu file helper** dan include di mana perlu:

auth.php

```

<?php
session_start();

// Cek apakah user sudah login
if (!isset($_SESSION['user_id'])) {
    header("Location: login.php");
    exit;
}

// Fungsi untuk halaman khusus admin
function wajib_admin() {
    if ($_SESSION['role'] !== 'admin') {
        die("Akses ditolak. Halaman ini hanya untuk admin.");
    }
}

```

```
}
}
?>
```

Cara pakai sangat sederhana, tambahkan satu baris di paling atas file yang ingin diproteksi contoh:

```
<?php
include 'auth.php';      // <- cek login dulu
include 'koneksi.php';

// ... kode halaman seperti biasa ...
?>
```

Untuk halaman yang khusus admin seperti **hapus.php**, panggil fungsi **wajib_admin()** setelah include:

```
<?php
include 'auth.php';
include 'koneksi.php';
wajib_admin();  // <- hentikan kalau bukan admin

// ... kode delete seperti biasa ...
?>
```

Selain proteksi server-side di atas, kita juga bisa menyembunyikan tombol/link tertentu dari user biasa di tampilan. Misalnya, di **tampil_data.php**, link "Hapus" hanya muncul untuk admin:

```
<?php if ($_SESSION['role'] === 'admin'): ?>
    <a href="hapus.php?id=?= $row['id']; ?>">Hapus</a>
<?php endif; ?>
```

TIPS: Selalu gunakan dua lapis proteksi, **sembunyikan UI** untuk kenyamanan pengguna, dan **validasi di server** untuk keamanan. Jangan andalkan UI saja, karena penyerang bisa tetap mengakses URL **hapus.php?id=x** langsung lewat browser.

TUGAS PRAKTIKUM

Buatlah sebuah aplikasi web menggunakan PHP dan MySQL dengan tema yang ditentukan sendiri tetapi harus memenuhi ketentuan minimum berikut:

- **Database minimal 2 tabel:** satu untuk autentikasi pengguna dan satu untuk data utama sesuai tema. Tabel data utama memiliki minimal 5 kolom dengan tipe data yang bervariasi.
- **Autentikasi dan Session:** terdapat fitur registrasi, login, dan logout. Password disimpan dalam bentuk hash, dan sesi pengguna dikelola dengan session.
- **Operasi CRUD pada Halaman Terproteksi:** implementasikan keempat operasi Create, Read, Update, dan Delete pada data utama. Seluruh halaman CRUD hanya dapat diakses setelah pengguna login, dan terapkan minimal 2 role (admin dan user) dengan kewenangan yang berbeda.
- **Keamanan dan Struktur Kode:** gunakan prepared statement pada seluruh query yang melibatkan input pengguna, gunakan `htmlspecialchars()` saat menampilkan data ke HTML, serta pisahkan file koneksi dan file pengecekan autentikasi dari halaman lain.
- **Tampilan dan Validasi:** gunakan framework CSS (Bootstrap atau TailwindCSS) untuk mempercantik tampilan, dan tambahkan validasi form sederhana di sisi browser.